COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	09 / 14 / 2026	**DATE SUBMITTED:**	
NAME:	Elijah I. Dizor	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 8.2 **SORTING ALGORITHMS: SHELL, MERGE, AND QUICK SORT**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code for implementation of shell, merge and quick sort.
A.1. Shell Sort

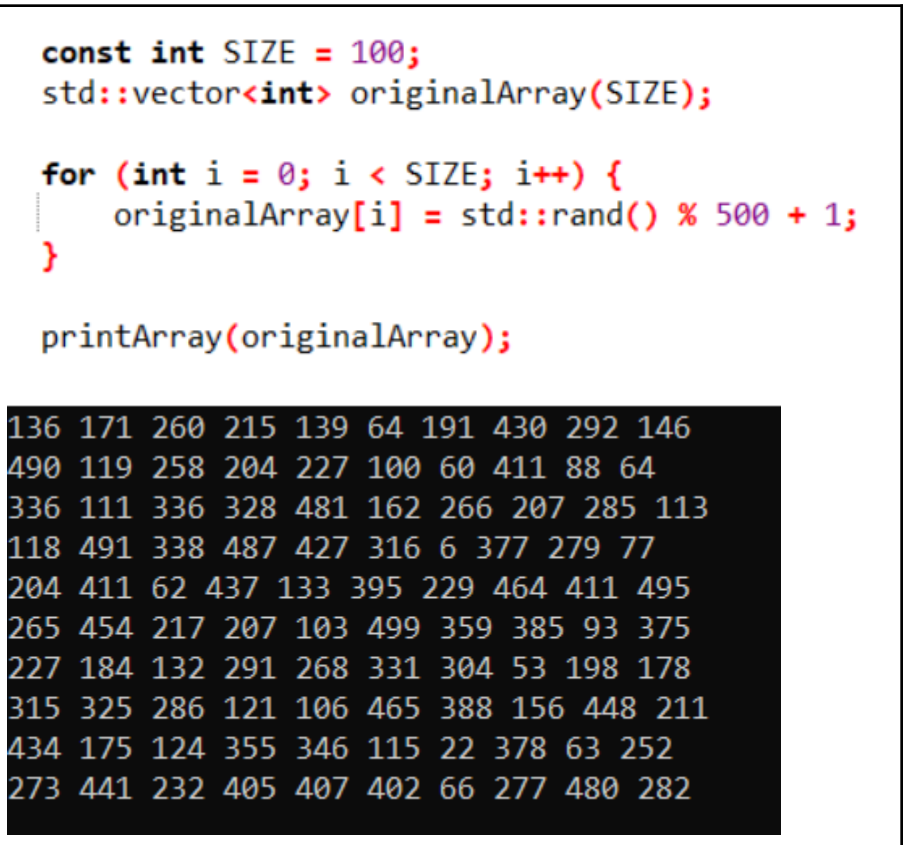
Code + Screenshot	
Observation	**The loop populates a 100 element with random numbers generated between 1 and 500. It then seeds the generator with the current time and the numbers change every time you run the code**

Table 8-1. Array of Values for Sort Algorithm Testing

A.2. Merge Sort

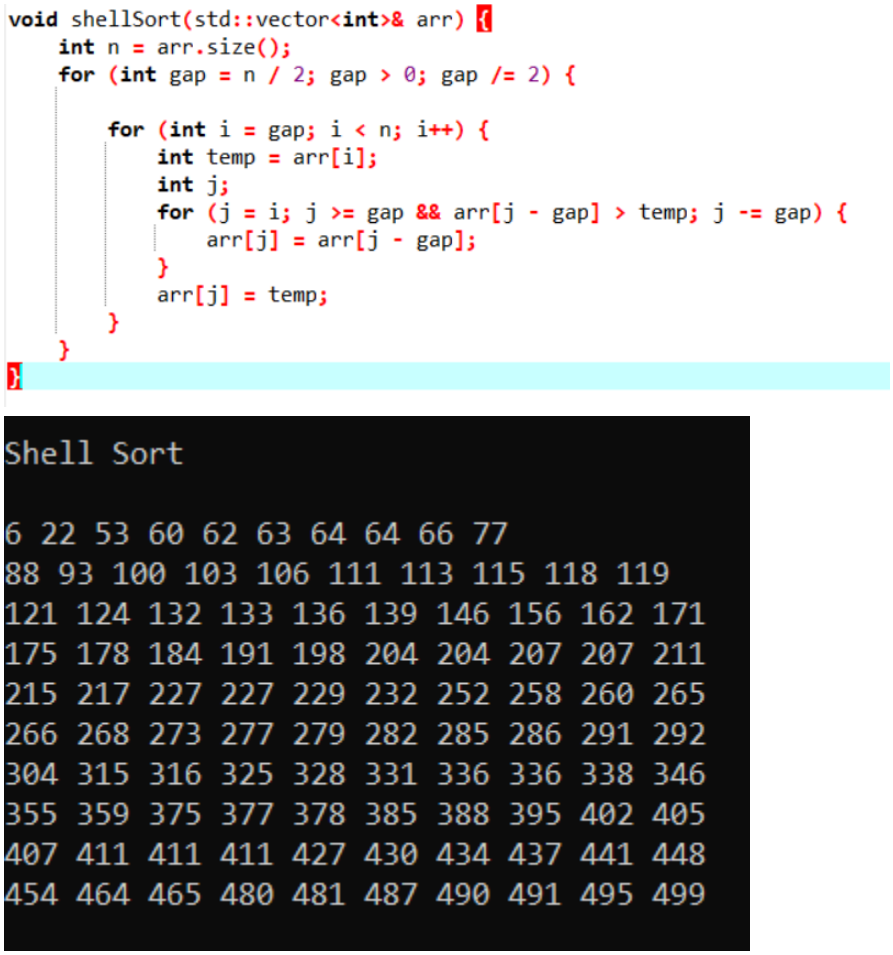
Code + Screenshot	
Observation	**The shell sort compare elements separated by a shifting gap, starting at n/2 and cuts in half each pass**

Table 8-2. Shell Sort Technique

A.3. Quick Sort

Code + Screenshot

```
void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSortHelper(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortHelper(arr, left, mid);
        mergeSortHelper(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void mergeSort(std::vector<int>& arr) {
    if (!arr.empty()) {
        mergeSortHelper(arr, 0, arr.size() - 1);
    }
}
```

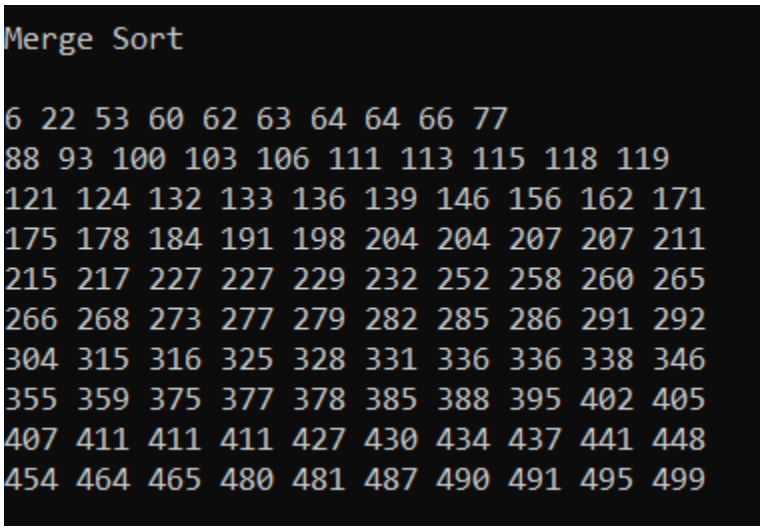
	
Observation	It recursively splits the array into single element subarrays, the merge function pieces them back together in order using temporary left and right arrays.

Table 8-3. Merge Sort Algorithm

Code + Screenshot	```
	int partition(std::vector<int>& arr, int low, int high) {
	    int pivot = arr[high];
	    int i = (low - 1);
	
	    for (int j = low; j < high; j++) {
	        if (arr[j] < pivot) {
	            i++;
	            std::swap(arr[i], arr[j]);
	        }
	    }
	    std::swap(arr[i + 1], arr[high]);
	    return (i + 1);
	}
	
	void quickSortHelper(std::vector<int>& arr, int low, int high) {
	    if (low < high) {
	        int pi = partition(arr, low, high);
	        quickSortHelper(arr, low, pi - 1);
	        quickSortHelper(arr, pi + 1, high);
	    }
	}
	
	void quickSort(std::vector<int>& arr) {
	    if (!arr.empty()) {
	        quickSortHelper(arr, 0, arr.size() - 1);
	    }
	}
	
	#endif
	``

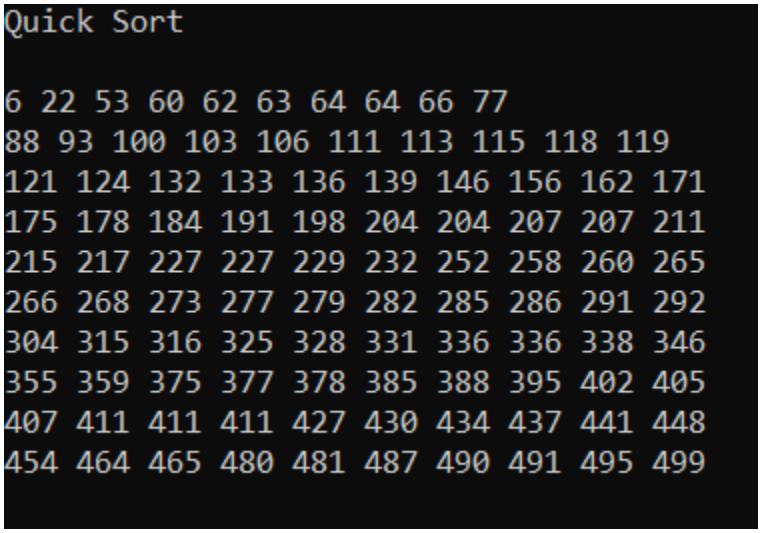
	
Observation	**It uses the final element as a structural pivot point. The partition function moves all numbers smaller than the pivot to the left side and larger numbers to the right side. It then repeats the process for the left and right partition**

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO B: Solve given data sorting problems using appropriate basic sorting algorithm

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

- It possible to use other sorting algorithms to sort the left and right sub-list generated after the quicksort partition

CODE

```cpp
#include <iostream>
#include <vector>
#include "sorting.h"

void printArray(const std::vector<int>& arr) {
    for (int val : arr) {
        std::cout << val << " ";
    }
}

int main() {

    std::cout << "Sub-List Sorting Pass\n\n";
    std::vector<int> dataset = {35, 12, 48, 25, 9, 60, 18};
    std::cout << "Initial Input Array: \n\n";
    printArray(dataset);
    std::cout<<"\n";
    solveProblem1(dataset);
    std::cout<<"\n";
    std::cout << "Final Sorted Array: \n\n";
    printArray(dataset);

    return 0;
}
```

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <vector>
#include <algorithm>

int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[high]);
    return (i + 1);
}

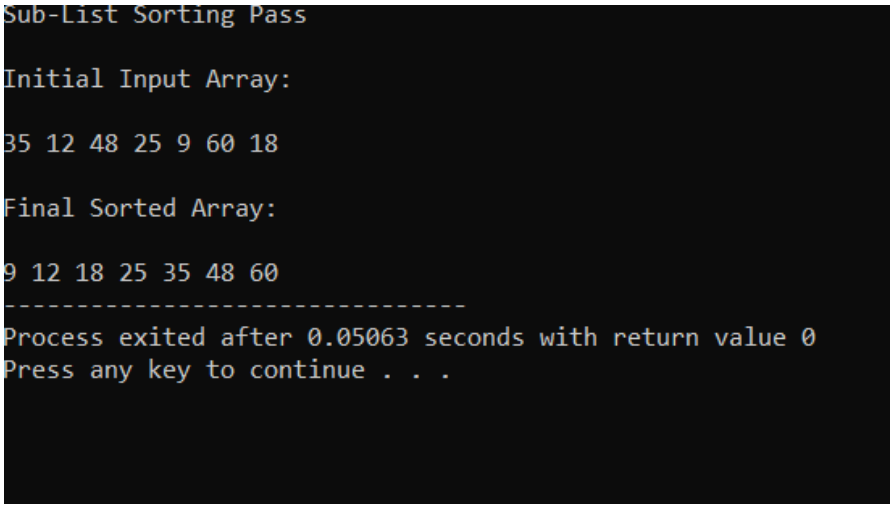
void bubbleSortSubList(std::vector<int>& arr, int low, int high) {
    for (int i = low; i <= high; i++) {
        for (int j = low; j < high - (i - low); j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

void solveProblem1(std::vector<int>& arr) {
    if (arr.size() < 2) return;

    int low = 0;
    int high = arr.size() - 1;

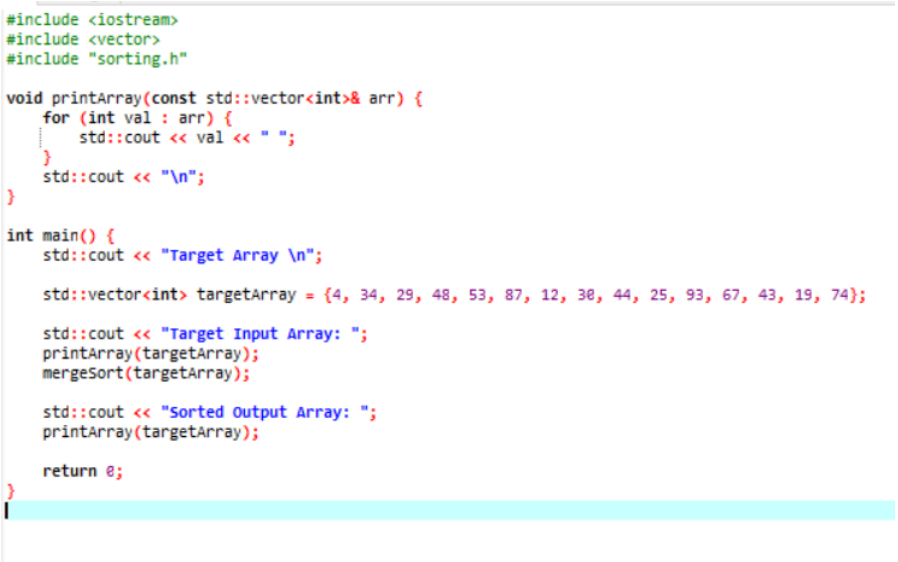
    int pivotIndex = partition(arr, low, high);
    bubbleSortSubList(arr, low, pivotIndex - 1);
    bubbleSortSubList(arr, pivotIndex + 1, high);
}

#endif
```

OUTPUT	
OBSERVATION	Partition function designates the final element of the array vector as a structural pivot. It then iterates through the dataset, shifting elements smaller than the pivot to the left index range and elements larger than the pivot to the right.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

- Quicksor is faster. The logN algorithm repeatedly splits the dataset in half, continuously dividing an array of size N by 2. The N factor scans all the N elements either to merge back together in order or to shift them around a pivot element

CODE	

```cpp
#include <iostream>
#include <vector>
#include "sorting.h"

void printArray(const std::vector<int>& arr) {
    for (int val : arr) {
        std::cout << val << " ";
    }
    std::cout << "\n";
}

int main() {
    std::cout << "Target Array \n";

    std::vector<int> targetArray = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};

    std::cout << "Target Input Array: ";
    printArray(targetArray);
    mergeSort(targetArray);

    std::cout << "Sorted Output Array: ";
    printArray(targetArray);

    return 0;
}
```

```cpp
#ifndef SORTING_H
#define SORTING_H

#include <vector>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    std::vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

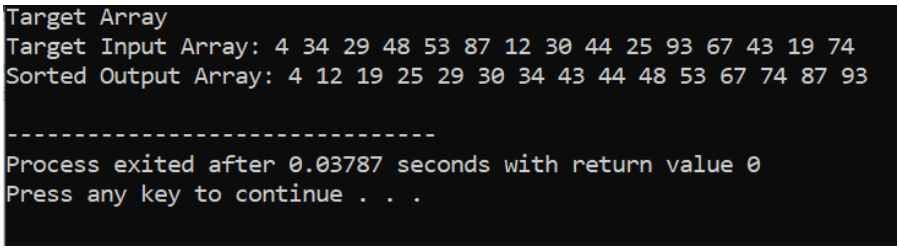
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSortHelper(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSortHelper(arr, left, mid);
        mergeSortHelper(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void mergeSort(std::vector<int>& arr) {
    if (!arr.empty()) {
        mergeSortHelper(arr, 0, arr.size() - 1);
    }
}

#endif
```

OUTPUT	
OBSERVATION	It calculates the midpoint index of the incoming array vector and splits it down to smaller sub-problems. The division repeats recursively until the data segments are broken down into single element lists that are being sorted out by default

Conclusion:

I learned the practical value of keeping data generation independent from algorithm mechanics, which made debugging and code adjustment much more manageable. The difference in structural approach impacts efficiency, divide and conquer strategies.

Based on my performance, I only managed to do the basic use and coding debugging of the three sorting algorithms. As for my areas for improvement, i still need to find better ways to implement the code and learn how to combine them all to able to solve complex problems

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.



Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.